LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 06 – INHERITANCE



NAMA: Nur Tsalits Alwan Mubarok

NIM: 103132400039

KELAS: S1SD-05-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Inheritance (pewarisan) adalah salah satu pilar utama pemrograman berorientasi objek yang memungkinkan sebuah kelas diturunkan dari kelas lain sehingga mewarisi atribut dan method-nya. Kelas yang menurunkan disebut superclass (kelas induk), sedangkan kelas yang diturunkan disebut subclass (kelas turunan). Hubungan keduanya bersifat "is-a", dan semakin ke bawah hierarkinya kelas menjadi semakin spesifik. Di Java, pewarisan dituliskan dengan keyword extends. Manfaat utamanya adalah menghindari duplikasi kode karena anggota yang sama cukup ditulis satu kali di superclass.

Keyword super dipakai oleh subclass untuk merujuk ke superclass. Keyword ini berguna untuk memanggil constructor superclass dengan super(...), memanggil method milik superclass dengan super.namaMethod(), serta mengakses atribut superclass. Pemanggilan constructor superclass harus berada di baris pertama constructor subclass. Atribut yang bersifat private tidak dapat diakses langsung oleh subclass, sehingga harus diakses melalui method seperti getter.

Overloading method adalah pembuatan dua atau lebih method dengan nama yang sama dalam satu kelas, tetapi dengan jumlah atau tipe parameter yang berbeda. Java menentukan method mana yang dipanggil berdasarkan argumen yang diberikan.

Overriding method terjadi ketika subclass mendefinisikan ulang method milik superclass dengan nama dan parameter yang sama. Jika method tersebut dipanggil dari objek subclass, maka versi milik subclass yang dijalankan. Versi milik superclass tetap dapat dipanggil menggunakan super.

Polymorphism memungkinkan variabel bertipe superclass menunjuk ke objek subclass. Method yang dijalankan ditentukan oleh tipe objek sebenarnya saat program berjalan, bukan oleh tipe variabelnya. Pewarisan juga dapat bersifat bertingkat (multilevel), yaitu subclass yang kembali menjadi superclass bagi kelas lain.

B. Guided

1. Inheritance

```
//file A.java

package guided1;

public class A {
    int x;
    int y;

    void tampilXY(){
        System.out.println("Nilai x = "+x+", y = "+y);
```

```
    }
}
```

```
//file B.java

package guided1;

public class B extends A{
    int z;
    int hasil;

    void jumlahXYZ(){
        hasil = (x+y+z);

        System.out.println("Jumlah (x+y+z))= " + hasil);
    }
}
```

```
//file Demo.java

package guided1;

public class Demo {

    public static void main(String[] args){

        A superclass = new A();
        B subclass = new B();

        superclass.x = 3;
        superclass.y = 4;

        System.out.println("Superclass: ");
        superclass.tampilXY();

        subclass.x = 1;
        subclass.y = 2;
        subclass.z = 5;

        System.out.println("Subclass: ");
        subclass.jumlahXYZ();
    }
}
```

Output:

```
Superclass:
Nilai x = 3, y = 4
Subclass:
Jumlah (x+y+z))= 8
```

2. Keyword Super

```
//file Demo.java

package guided2;

public class Demo {
    public static void main(String[] args){
        Manager jarwo = new Manager("Jarwo","sukabirus");
        jarwo.departemen = "SDM";
        jarwo.tampilInformasi();
    }
}
```

```
//file Employee.java

package guided2;

public class Employee {
    private String nama;
    String departemen;

    public Employee(String nama){
        this.nama = nama;
    }

    public void tampilNama(){
        System.out.println("nama :"+ nama);
    }
}
```

```
//file Manager.java

package guided2;

public class Manager extends Employee{
    private String alamat;

    public Manager(String nama, String alamat){
        super(nama);
```

```
        this.alamat = alamat;
    }

    public void tampilInformasi(){
        super.tampilNama();
        System.out.println("alamat: " + alamat);
        System.out.println("departemen: " + super.departemen);
    }
}
```

Output:

```
nama :Jarwo
alamat: sukabirus
departemen: SDM
```

3. Method Overloading

```
//file lagu.java

package guided3;

public class lagu {
    String pencipta;
    String judul;

    void IsiParam(String param1) {
        judul = param1;
        pencipta = "Tidak dikenal";
    }

    void IsiParam(String param1, String param2) {
        judul = param1;
        pencipta = param2;
    }

    void tampilkan() {
        System.out.println("Judul : " + judul + ", pencipta : " + pencipta);
    }
}
```

```
//file main.java

package guided3;
```

```
public class main {
    public static void main(String[] args) {
        lagu d, e;
        d = new lagu();
        e = new lagu();

        d.IsiParam("Lagu 1");
        e.IsiParam("kepastian yang kutunggu", "GiGi");

        d.tampilkan();
        e.tampilkan();
    }
}
```

Output:

```
Judul : Lagu 1, pencipta : Tidak dikenal
Judul : kepastian yang kutunggu, pencipta : GiGi
```

4. Method Overriding

```
//file A.java

package guided4;

public class A {
    public void tampilkanKeLayar() {
        System.out.println("Method milik class A dipanggil...");
    }
}
```

```
//file B.java

package guided4;

public class B extends A{
    public void tampilkanKeLayar() {
        super.tampilkanKeLayar();
        System.out.println("Method milik class B dipanggil...");
    }
}
```

```
//file Main.java

package guided4;
```

```
public class Main {
    public static void main(String[] args) {
        B subOb = new B();
        subOb.tampilkanKeLayar();
    }
}
```

Output:

```
Method milik class A dipanggil...
Method milik class B dipanggil...
```

C. Unguided

1) Sebuah tim machine learning ingin membuat program untuk membandingkan estimasi biaya komputasi dari beberapa jenis model yang akan dilatih. Ada tiga jenis model: model regresi, model klasifikasi, dan model klasifikasi berbasis deep learning yang merupakan pengembangan dari model klasifikasi biasa.
   Buatlah program Java dengan ketentuan: Penamaan kelas, atribut, dan method ditentukan sendiri oleh Anda dan semua atribut bersifat private.

Jawaban:

```
//file ModelML.java

public class ModelML {
    private String namaModel;
    private int jumlahDataLatih;

    public ModelML(String namaModel, int jumlahDataLatih) {
        this.namaModel = namaModel;
        this.jumlahDataLatih = jumlahDataLatih;
    }

    public int getJumlahDataLatih() {
        return jumlahDataLatih;
    }

    public void tampilkanInfo() {
        System.out.println("Model: " + namaModel + " | Data latih: " + jumlahDataLatih);
    }

    // Overloading 1: tanpa parameter
    public int hitungBiaya() {
        return jumlahDataLatih * 2;
```

```
    }

    // Overloading 2: dengan parameter epoch
    public int hitungBiaya(int epoch) {
        return jumlahDataLatih * epoch;
    }
}
```

```
//file ModelRegresi.Java

public class ModelRegresi extends ModelML {
    private int jumlahFitur;

    public ModelRegresi(String namaModel, int jumlahDataLatih, int jumlahFitur) {
        super(namaModel, jumlahDataLatih);
        this.jumlahFitur = jumlahFitur;
    }

    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah fitur: " + jumlahFitur);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() + (jumlahFitur * 10);
    }

    // Overloading 3: epoch dan ukuran batch
    public int hitungBiaya(int epoch, int ukuranBatch) {
        return getJumlahDataLatih() * epoch / ukuranBatch;
    }
}
```

```
//file ModelKlasifikasi.java

public class ModelKlasifikasi extends ModelML {
    private int jumlahKelas;

    public ModelKlasifikasi(String namaModel, int jumlahDataLatih, int jumlahKelas) {
        super(namaModel, jumlahDataLatih);
        this.jumlahKelas = jumlahKelas;
    }

    @Override
```

```
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah kelas: " + jumlahKelas);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() * jumlahKelas;
    }
}
```

```
//file ModelDeepLearning.java

public class ModelDeepLearning extends ModelKlasifikasi {
    private int jumlahLayer;

    public ModelDeepLearning(String namaModel, int jumlahDataLatih, int jumlahKelas,
int jumlahLayer) {
        super(namaModel, jumlahDataLatih, jumlahKelas);
        this.jumlahLayer = jumlahLayer;
    }

    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah layer: " + jumlahLayer);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() + (jumlahLayer * 200);
    }
}
```

```
//file Main.java

public class Main {
    public static void main(String[] args) {
        ModelRegresi regresi = new ModelRegresi("Regresi Linier", 500, 4);
        ModelKlasifikasi tree = new ModelKlasifikasi("Decision Tree", 800, 3);
        ModelDeepLearning nn = new ModelDeepLearning("Neural Network", 1000, 5, 3);

        ModelML[] daftarModel = {regresi, tree, nn};

        int totalBiaya = 0;
        for (ModelML m : daftarModel) {
```

```
            m.tampilkanInfo();
            int biaya = m.hitungBiaya();
            System.out.println("Biaya komputasi: " + biaya);
            System.out.println("---");
            totalBiaya += biaya;
        }
        System.out.println("Total biaya: " + totalBiaya);

        System.out.println("Biaya 10 epoch (Neural Network): " +
daftarModel[2].hitungBiaya(10));
        System.out.println("Biaya 10 epoch (Regresi Linier): " +
daftarModel[0].hitungBiaya(10));

        // Method 2 parameter hanya ada di ModelRegresi, jadi perlu casting
        ModelRegresi r = (ModelRegresi) daftarModel[0];
        System.out.println("Biaya 10 epoch batch 50 (Regresi Linier): " + r.hitungBiaya(10,
50));
    }
}
```

Output:

```
Model: Regresi Linier | Data latih: 500
Jumlah fitur: 4
Biaya komputasi: 1040
---
Model: Decision Tree | Data latih: 800
Jumlah kelas: 3
Biaya komputasi: 4800
---
Model: Neural Network | Data latih: 1000
Jumlah kelas: 5
Jumlah layer: 3
Biaya komputasi: 10600
---
Total biaya: 16440
Biaya 10 epoch (Neural Network): 10000
Biaya 10 epoch (Regresi Linier): 5000
Biaya 10 epoch batch 50 (Regresi Linier): 100
```

Penjelasan:

ModelML adalah kelas induk dengan atribut private namaModel dan jumlahDataLatih. Kelas ini memiliki constructor, getter getJumlahDataLatih(), method tampilkanInfo(), dan dua method hitungBiaya() yang merupakan overloading: tanpa parameter menghitung data × 2, sedangkan dengan parameter epoch menghitung data × epoch.

ModelRegresi mewarisi ModelML dan menambah atribut jumlahFitur. Constructor-nya memanggil super(...) untuk mengisi nama dan data latih. Method tampilkanInfo() dan hitungBiaya() di-override, dengan biaya menjadi biaya induk + (fitur × 10). Kelas ini juga menambah hitungBiaya(epoch, ukuranBatch) sebagai overloading ketiga dengan rumus data × epoch ÷ batch, dan data latih diambil lewat getter karena atributnya private.

ModelKlasifikasi mewarisi ModelML dan menambah atribut jumlahKelas. Method tampilkanInfo() dan hitungBiaya() di-override, dengan biaya menjadi biaya induk × jumlah kelas. Versi hitungBiaya(epoch) tidak di-override sehingga memakai versi milik ModelML.

ModelDeepLearning mewarisi ModelKlasifikasi (pewarisan bertingkat) dan menambah atribut jumlahLayer. Constructor-nya memanggil super(...) milik ModelKlasifikasi. Biaya dihitung dari super.hitungBiaya() milik ModelKlasifikasi, lalu ditambah (layer × 200).

Pada class Main, ketiga objek disimpan dalam satu array bertipe ModelML sehingga dapat diproses bersama dalam satu perulangan (polymorphism). Setiap pemanggilan tampilkanInfo() dan hitungBiaya() otomatis menjalankan versi milik objek aslinya. Hasilnya: Regresi Linier 1040, Decision Tree 4800, Neural Network 10600, dengan total biaya 16440. Untuk 10 epoch didapat 10000 (Neural Network) dan 5000 (Regresi Linier), sedangkan 10 epoch dengan batch 50 pada Regresi Linier menghasilkan 100. Pemanggilan versi dua parameter memerlukan casting ke ModelRegresi karena method tersebut hanya ada di kelas itu.

Kesimpulan

Berdasarkan praktikum Modul 6, inheritance memungkinkan subclass mewarisi atribut dan method dari superclass sehingga kode tidak perlu ditulis berulang. Keyword super digunakan untuk memanggil constructor, method, dan atribut milik superclass.

Overloading membuat satu nama method dapat dipakai dengan parameter yang berbeda dalam satu kelas, sedangkan overriding memungkinkan subclass mengubah

perilaku method warisan dan tetap dapat memanggil versi superclass lewat super. Pada unguided, pewarisan bertingkat dan array bertipe superclass menunjukkan bahwa polymorphism membuat program lebih ringkas, mudah dikembangkan, dan mudah dipelihara.